

Tensors, Datasets & Dataloaders

Sattam Altwaim

sattam.tuuaim@kaust.edu.sa



جامعة الملك عبد الله
للعلوم والتقنية
King Abdullah University of
Science and Technology



KAUST Academy & Mawhiba
AI Olympiad Training (Beginners)
Week 3, Day 19, Self-study

7 October 2026

1. Where This Fits
2. Learning Outcomes
3. Tensors in Depth
4. Datasets in Depth
5. DataLoaders in Depth
6. Olympiad Practice
7. Check Yourself
8. Limitations and What Comes Next
9. Summary
10. References

- ▶ **In the lectures:** Day 15 said a tensor is a NumPy array with autograd and a GPU switch, showed shapes in code and fixed the rule “features float32, class labels int64”. Day 16 defined a Dataset as a list of (x, y) pairs with `__len__` and `__getitem__`, a DataLoader as the thing that shuffles, cuts and stacks them, wrote one Dataset class that reads files, and listed the dials. The labs used `random_split` on MNIST, a BeatDataset with standardisation and a CSV turned into a TensorDataset.
- ▶ **This deck adds:** the three labels of a tensor and the error each one throws; shape surgery and broadcasting with numbers; the memory arithmetic; the Dataset contract with a checklist; eager against lazy loading, timed; transforms and splits without leakage; what one loader iteration does; what shuffle, workers and batch size cost or buy, timed; the loader errors and their fixes.
- ▶ **How to use it:** every frame stands alone and every number was measured. Run each code frame in a notebook, break it on purpose, and read the message you get back.

Where the simple version breaks

- ▶ Day 16's loop runs an epoch of 20,000 MNIST images in 0.30 s when the tensors sit in memory. The same loop over the same images stored as 20,000 PNG files takes 1.85 s with no workers, and 8 s per epoch once each file is resized to 224 by 224 the way Week 4 will. The loop would spend its day waiting for the disk.
- ▶ A table that arrives as `float64`, a label column that arrives as `float32`, a target of shape `[64]` against an output of shape `[64, 1]`: each stops the loop with a message, and the message says exactly what to change.
- ▶ Forgetting `shuffle=True` on data sorted by label leaves the MNIST test accuracy at 0.195 instead of 0.923 after three epochs. Nothing crashes; the dashboard just lies.

After this deck you can

build the data side of any training run so that the shapes, dtypes, devices and speed are right before the first epoch starts, and read every data error PyTorch throws.

By the end of this self-study deck, you will be able to:

- ▶ **Read** a tensor's shape, dtype and device, and fix the error each one throws.
- ▶ **Reshape**, flatten, stack and broadcast tensors with the shapes written out at every step.
- ▶ **Estimate** the memory of a dataset, a batch and a model from the element count and the bytes per element.
- ▶ **Write** a Dataset class that keeps the contract, chooses eager or lazy loading on purpose, and applies transforms and splits without leakage.
- ▶ **Configure** a DataLoader (batch size, shuffle, workers, drop_last) and say what each setting costs or buys.
- ▶ **Diagnose** a loader or collate error from its message and fix the Dataset, not the loop.

Tensors: **Tensors in Depth**

Three Labels: Shape, dtype, Device

```
x = torch.tensor(df.values,  
                 dtype=torch.float32)  
x.shape          # torch.Size([2400, 23])  
x.dtype          # torch.float32  
x.device         # device(type='cpu')  
x = x.to("cuda")  # now on the GPU  
x.ndim, x.numel() # 2, 55200
```

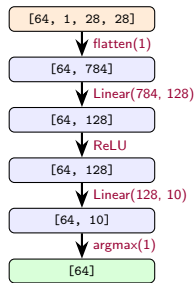
The rule

Every operation needs its tensors to agree on all three: shapes that fit the operation, the same dtype, the same device. Each disagreement has its own message, and the message names the two sides.

Message	Fix
mat1 and mat2 shapes cannot be multiplied (64x784 and 128x10): the batch has 784 columns, the layer reads 128	compare <code>xb.shape[1]</code> with <code>model[0].in_features</code> ; flatten the images first
mat1 and mat2 must have the same dtype, but got Double and Float: a NumPy float64 table met float32 dials	<code>torch.tensor(a, dtype=torch.float32)</code> or <code>x.float()</code>
Expected all tensors to be on the same device, but found at least two devices, cuda:0 and cpu!	<code>xb.to(device)</code> inside the loop; <code>.cpu()</code> before NumPy

```
b = torch.rand(64, 1, 28, 28)      # N, C, H, W
b.flatten(1).shape                 # [64, 784]
b.reshape(64, -1).shape            # [64, 784], -1: work it out
b.squeeze(1).shape                 # [64, 28, 28]
b.squeeze(1).unsqueeze(1).shape    # [64, 1, 28, 28]
hwc = torch.rand(64, 28, 28, 3)    # PIL order H, W, C
hwc.permute(0, 3, 1, 2).shape       # [64, 3, 28, 28]
torch.stack([b[0], b[1]]).shape     # [2, 1, 28, 28]
torch.cat([b, b]).shape             # [128, 1, 28, 28]
```

- ▶ **Batch first, always:** dim 0 counts samples; every layer and loss assumes it. reshape and flatten relabel the same numbers, permute moves them.
- ▶ stack makes a new dimension from equal pieces (what the DataLoader does to 64 samples); cat glues along an existing one.



one batch through the 784-128-10 network:
the first number never changes

```
X = torch.tensor([[1., 10.],  
                  [2., 20.],  
                  [3., 30.]]) # [3, 2]  
m = X.mean(dim=0) # [2., 20.], shape [2]  
X - m # [[-1, -10], [0, 0], [1, 10]]  
X - X.mean(dim=1) # minus [3]: ERROR  
X - X.mean(1, keepdim=True) # [3, 1]: ok
```

The rule

Shapes are aligned from the right. A dimension of size 1, or a missing one, is stretched to match; any other mismatch is an error: The size of tensor a (2) must match the size of tensor b (3) at non-singleton dimension 1.

Left	Right	Result
[64, 784]	[784]	[64, 784]: one mean per column
[64, 784]	[64, 1]	[64, 784]: one per row
[64, 784]	[64]	error
[64, 1]	[64]	[64, 64], silently

The silent one

An output of shape [64, 1] minus a target of shape [64] is a 64 by 64 matrix, and its mean is a wrong loss with no error. Losses check this (Target size ... must be the same as input size); your own code does not. `out.squeeze(1)` first.

What	dtype	Shape
features; regression targets	float32	[N, d]; [N]
class labels for CrossEntropyLoss	int64	[N]
yes/no targets for BCEWithLogitsLoss	float32	the output's shape, [N, 1]
images on disk and in RAM	uint8, 0 to 255	float32 only inside the batch
masks	bool	any

```
t = torch.tensor(a)           # a copy; float64 if a is
t = torch.from_numpy(a)       # shares memory with a
t = torch.tensor(a, dtype=torch.float32) # the usual line
y = torch.tensor(df["label"].values)    # int64 from an int column
a = t.detach().cpu().numpy()            # back: no grad, on the CPU
v = loss.item()                        # one Python number
t = t.to(device); t = t.cpu()          # the device switch
```

- ▶ `torch.tensor(np_array)` keeps NumPy's float64: that is where “Double and Float” comes from. Say the dtype. `CrossEntropyLoss` with float32 labels says expected scalar type Long but found Float: labels are classes, so int64.
- ▶ Log with `.item()`. A list of loss tensors keeps every graph alive and eats the memory of the run.

bytes = number of elements $\times$ bytes per element (uint8 : 1, float32 : 4, int64 : 8, float64 : 8)

Tensor	Elements	Size
MNIST, $60,000 \times 784$, uint8	47.0 M	47 MB
the same as float32	47.0 M	188 MB
CIFAR-10, $50,000 \times 3 \times 32 \times 32$, uint8 / float32	153.6 M	154 MB / 614 MB
one batch of 64 photos, $3 \times 224 \times 224$, float32	9.6 M	38.5 MB
the 784-128-10 network, float32	101,770	0.4 MB

- ▶ The data is the big thing, not the model. Keep images uint8 on disk and in memory, and turn one batch at a time into float32: four times less RAM for the same pictures.
- ▶ A GPU holds the batch, the dials and every activation of the forward pass. CUDA out of memory is not a bug to fix in the model: halve the batch size, or feed smaller images.
- ▶ `x.numel() * x.element_size()` prints the bytes of any tensor; do the sum once before a long run.

Tensors: **Datasets in Depth**

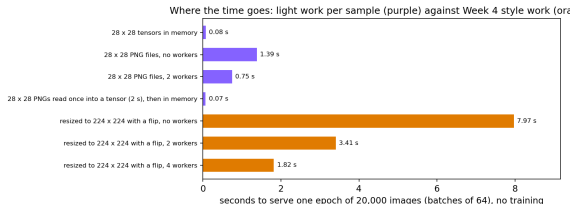
```
class Pairs(Dataset):
    def __init__(self, X, y):  # store, not
                               # compute
        self.X = torch.tensor(X).float()
        self.y = torch.tensor(y).long()
    def __len__(self):
        return len(self.y)
    def __getitem__(self, i):  # one sample
        return self.X[i], self.y[i]

def check(ds):
    x, y = ds[0]
    print(len(ds), x.shape, x.dtype, y.dtype)
    loader = DataLoader(ds, batch_size=8)
    xb, yb = next(iter(loader))
    print(xb.shape, yb.shape)  # [8, ...] and
                               [8]
```

The contract

$\text{len}(ds)$ is N . For every $0 \leq i < N$, $ds[i]$ returns one sample as tensors, with the **same shape and dtype every time**. Nothing else is required.

- ▶ The loader stacks 64 samples with `torch.stack`: equal shapes or the collate error of the last frame. A label as a 0-dim int64 tensor stacks into [64].
- ▶ Run `check(ds)` before any loop. Ten seconds here save the hour spent reading the first error at epoch 0.



Source: the 20,000 training images of the Day 15 MNIST split, once as a `TensorDataset` and once as 20,000 PNG files read by a `Dataset` class, batches of 64, one epoch each on a 10-core laptop; `scripts/day19_data_figures.py`.

- ▶ Light work per sample (purple): serving 20,000 images from a tensor takes 0.08 s; opening 20,000 PNG files takes 1.39 s, two workers bring it to 0.75 s, and reading the files once into a tensor (1.6 s) makes every later epoch cost 0.07 s. Small images that fit in memory: read once and keep.
- ▶ Week 4 style work (orange): decode, resize to 224 by 224 and flip, 7.97 s per epoch on one process, 3.41 s with two workers, 1.82 s with four. Heavy work per sample, or data that does not fit: the `Dataset` stays lazy and the workers carry it.

```
class Images(Dataset):
    def __init__(self, paths, y, m, s, tf=None):
        self.paths, self.y = paths, y
        self.m, self.s, self.tf = m, s, tf
    def __len__(self):
        return len(self.paths)
    def __getitem__(self, i):
        img = Image.open(self.paths[i])
        x = torch.tensor(np.array(img) / 255.0,
                        dtype=torch.float32)
        x = (x - self.m) / self.s      # train stats
        if self.tf is not None:
            x = self.tf(x)           # Week 4
        return x.unsqueeze(0), self.y[i]
```

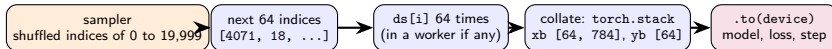
- ▶ A transform is anything applied inside `__getitem__`: it runs every time a sample is asked for, 20,000 times per epoch. The sample leaves as `[1, 28, 28]`, standardised with the training split's `m` and `s`.
- ▶ Fixed work (decode, resize, standardise) can be done once in `__init__` when the result fits in memory; random work (Week 4's augmentations) stays here, so each epoch sees a new version.
- ▶ torchvision:
`Compose([ToTensor(),
Normalize((0.1307,),
(0.3081,))])`, the MNIST mean and spread; `ToTensor` also maps `uint8 0 to 255 to float32 0 to 1`.

```
g = torch.Generator().manual_seed(0) # the same split every run
train_set, val_set = random_split(ds, [16000, 4000], generator=g)
tr, va = train_test_split(np.arange(len(ds)), test_size=0.2,
                           stratify=y, random_state=0)
train_set, val_set = Subset(ds, tr), Subset(ds, va)
mean, std = X[tr].mean(0), X[tr].std(0) # training rows only
train_loader = DataLoader(train_set, batch_size=64, shuffle=True)
val_loader = DataLoader(val_set, batch_size=1000) # no shuffle
```

- ▶ The seed makes the split the same on every run; without it, yesterday's checkpoint is scored on rows it trained on and the number means nothing.
- ▶ `stratify=y` keeps the class shares equal in both parts (Day 16's beats: 70% normal). `Subset` turns a list of indices into a Dataset without copying anything.
- ▶ Standardising with the mean of all rows leaks the validation rows into training. Compute on `X[tr]`, apply to both, as Day 16's fold loop did.

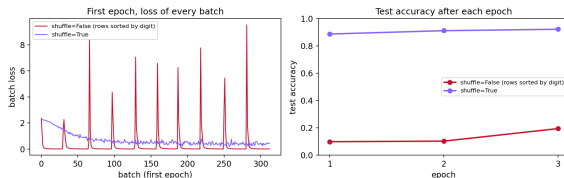
Tensors: **DataLoaders** in Depth

One Iteration, Step by Step



```
loader = DataLoader(ds, batch_size=64, shuffle=True)
len(loader)           # ceil(20000 / 64) = 313; drop_last: 312
xb, yb = next(iter(loader)) # one batch: look at shapes and dtypes
for step, (xb, yb) in enumerate(loader): # 313 steps = one epoch
    ...
```

- The loader owns no data: it asks the sampler for indices and the Dataset for samples, then stacks them; everything slow or wrong lives in `__getitem__`. `shuffle=True` reshuffles every epoch; the validation loader keeps its order so the score is the same number every run.



Source: the 20,000 MNIST training images sorted by digit, 784-128-10 network, SGD at 0.1, batches of 64, three epochs, with and without `shuffle=True`; `scripts/day19_data_figures.py`.

- ▶ Without shuffling every batch holds one digit: the loss drops to 0 inside each block and spikes at every change of digit (left). The network learns only the current digit and forgets the rest: after three epochs it answers 9 for 8,322 of 10,000 test images, accuracy 0.195.
- ▶ With `shuffle=True` the same network, data and dials reach 0.888, 0.912 and 0.923; the noisier batch loss is the point, every batch being a small sample of the whole table.
- ▶ Sorted data is the normal case (files by folder, a CSV by class or date): always shuffle the training loader, never the validation one.

```
loader = DataLoader(  
    ds, batch_size=64, shuffle=True,  
    num_workers=2,      # Colab has 2 CPUs  
    pin_memory=True)    # faster GPU copies  
  
if __name__ == "__main__": # scripts only  
    train(loader)
```

- ▶ Workers pay only when `__getitem__` does real work: on the 224 by 224 pipeline two workers cut an epoch from 7.97 s to 3.41 s and four to 1.82 s; on 28 by 28 PNGs the gain is 1.39 s to 0.75 s; on tensors in memory they only add cost.
- ▶ Workers are processes: they re-import your file, so the Dataset class sits at module level and the loop under the `__main__` guard. Notebooks on Linux need neither.

- ▶ `pin_memory=True` keeps each batch in page-locked RAM so the copy to the GPU does not wait; it does nothing on a CPU run.
- ▶ The batch size trades steps for speed (Day 16: 16 rows, 1,250 steps and 0.958 after three epochs; 1,024 rows, 20 steps and 0.893) and sets the memory: the batch, the dials and every activation must fit the GPU at once.
- ▶ CUDA out of memory: halve the batch; the learning rate may need to follow (Day 17). Validation batches can be ten times larger because nothing is kept for a backward pass.

Message	Cause	Fix
stack expects each tensor to be equal size, but got [3] at entry 0 and [4] at entry 1	<code>__getitem__</code> returned samples of different shapes	resize or pad inside <code>__getitem__</code> ; check <code>ds[i].shape</code> for a few <i>i</i>
mat1 and mat2 shapes cannot be multiplied (64x784 and 128x10)	the batch's columns do not match the first layer	<code>flatten(1)</code> in the Dataset or the model; compare the two numbers
mat1 and mat2 must have the same dtype, but got Double and Float	float64 from NumPy or pandas	<code>dtype=torch.float32</code> when building the tensor
expected scalar type Long but found Float	class labels stored as float	<code>dtype=torch.int64</code> for the labels; only BCEWithLogitsLoss wants float
Target size (torch.Size([64])) must be the same as input size (torch.Size([64, 1]))	yes/no target [N] against output [N, 1]	<code>y.unsqueeze(1)</code> in the Dataset, or <code>out.squeeze(1)</code>
Expected all tensors to be on the same device ... cuda:0 and cpu!	the batch or the model stayed on the CPU	<code>xb.to(device)</code> , <code>model.to(device)</code> , once each
DataLoader worker (pid ...) is killed by signal	workers ran out of RAM (eager data copied per worker)	<code>num_workers=0</code> to see the real error; load lazily or cache once

The pattern: the message names the two sides that disagree. Print both (`xb.shape`, `xb.dtype`, `xb.device` against the layer or the loss) and change the Dataset, not the loop.

Tensors: Olympiad Practice

Task (adapted)

`data_1.npz` and `data_2.npz` hold NumPy arrays `X1` of shape $(3840, 128, 4)$ and `X2` of shape $(3840, 4, 128)$, `float32`. Picture i is the matrix product `X1[i] @ X2[i]`, a 128×128 grey image. The pictures are altered copies of 32 originals, to be grouped back into 32 clusters with Week 5's tools; today's part is the data side.

```
X1 = np.load("data_1.npz")["arr_0"]; X2 = np.load("data_2.npz")["arr_0"]
X1.shape, X2.shape, X1.dtype          # (3840, 128, 4) (3840, 4, 128) float32
```

- ▶ **Question 1:** write the one line that turns the two arrays into a tensor of all 3,840 pictures, with its shape. What must one picture's shape become for a network that reads $[N, C, H, W]$?
- ▶ **Question 2:** how many megabytes is that tensor, and how many are the two arrays? Write a Dataset that makes one picture per index on demand, so the big tensor never exists.
- ▶ **Question 3:** the pixel values are not between 0 and 1. What do you do about it, where, and what does `check(ds)` print?

```
A = torch.from_numpy(X1)    # [3840, 128, 4]
B = torch.from_numpy(X2)    # [3840, 4, 128]
pics = torch.bmm(A, B)      # [3840, 128, 128]
```

```
class Pictures(Dataset):
    def __init__(self, A, B):
        self.A, self.B = A, B    # 15.7 MB
        self.lo = pics.min()
        self.rng = pics.max() - pics.min()
    def __len__(self):
        return len(self.A)
    def __getitem__(self, i):
        img = self.A[i] @ self.B[i]
        img = (img - self.lo) / self.rng
        return img.unsqueeze(0)
```

```
ds = Pictures(A, B); check(ds)
loader = DataLoader(ds, batch_size=64)
```

1. `torch.bmm` multiplies 3,840 pairs of matrices at once: $(3840, 128, 4) @ (3840, 4, 128)$ gives $[3840, 128, 128]$. A network wants a channel dimension: `unsqueeze(1)` makes $[3840, 1, 128, 128]$; a grey picture repeated three times (`repeat(1, 3, 1, 1)`) feeds a model trained on colour (Week 5).
2. $3,840 \times 128 \times 128 \times 4$ bytes = 252 MB; the two factor arrays are $2 \times 3,840 \times 128 \times 4 \times 4$ bytes = 15.7 MB. Multiplying inside `__getitem__` keeps the 15.7 MB and makes one $[128, 128]$ picture per index, 64 at a time.
3. Scale with the minimum and range of the whole set (computed once) inside `__getitem__`, and return $[1, 128, 128]$. There are no labels, so `ds[i]` is one tensor and `check(ds)` prints `3840 torch.Size([1, 128, 128]) torch.float32` and a batch of `[64, 1, 128, 128]`. No shuffle: the submission is one row per index, in order.

Tensors: **Check Yourself**

Answer before turning the page.

1. **Definition.** A batch `b` has shape `[64, 3, 32, 32]` and dtype `uint8`. What are the shape and size in bytes of `b.flatten(1)`, and of `b.flatten(1).float()`? Which of the two should the Dataset keep in memory for 50,000 such images, and why?
2. **Mechanism.** A loader over 1,000 samples with `batch_size=64`, `shuffle=True`, `drop_last=False`: how many batches per epoch, how big is the last one, what changes with `drop_last=True`, and why does the validation loader use neither shuffling nor `drop_last`?
3. **Code.** Each line of this Dataset hides one error that appears only when the loop starts. Name the message each one produces and the fix.

```
def __getitem__(self, i):  
    x = torch.tensor(self.table[i])           # self.table is a NumPy float64  
        array                                #  
    y = float(self.labels[i])                 # for nn.CrossEntropyLoss  
    return x[: self.lengths[i]], y           # rows have different lengths
```

1. `b.flatten(1)` is `[64, 3072]`: $64 \times 3,072 = 196,608$ elements, 196,608 bytes as `uint8` and 786,432 bytes as `float32` (four per element). For 50,000 images: 154 MB against 614 MB. Keep the `uint8` version and convert each batch to `float32` on its way out of `__getitem__` or just before the model.
2. $\lceil 1,000/64 \rceil = 16$ batches, the last one with $1,000 - 15 \times 64 = 40$ samples. `drop_last=True` gives 15 batches and never shows 40 of the samples in that epoch (a different 40 each epoch, because of the shuffle). Validation needs every sample exactly once, in a fixed order, so the score is one reproducible number.
3. Line 1: `torch.tensor` of a `float64` array gives a `float64` tensor: `mat1` and `mat2` must have the same dtype, but got `Double` and `Float`; add `dtype=torch.float32`. Line 2: a Python float label stacks into a `float32` batch, and `CrossEntropyLoss` wants `int64` classes: expected scalar type `Long` but found `Float`; use `torch.tensor(self.labels[i], dtype=torch.int64)` or store the labels as an `int64` tensor. Line 3: samples of different lengths: `stack` expects each tensor to be equal size; pad or cut every row to one length inside `__getitem__`.

Tensors: **Limitations & What Comes Next**

- ▶ **The timings are one laptop's.** Ten cores, a fast disk, 28 by 28 images. Colab has two CPUs and a slower disk, so the file-reading bars grow and the worker gain shrinks; the ordering of the bars is what carries over.
- ▶ **Workers have a fixed cost.** Starting processes and copying batches back costs about a second per epoch; on tiny datasets in memory, `num_workers=0` is the fastest setting.
- ▶ **Everything here fits in RAM or on one disk.** Data that streams from the network or does not fit on the machine uses an `IterableDataset`, named here and not taught.
- ▶ **The memory numbers assume float32.** Training in half precision halves them; it comes with the GPU weeks, not now.

The common mistake

Treating a data error as a model error. “Double and Float”, “cannot be multiplied”, “equal size”, “same device” all come from the Dataset or the loader. Read the two sides the message names, run `check(ds)`, fix `__getitem__`.

- ▶ **Day 20, self-study, Olympiad practice:** reading loss curves, the optimizer and regularisation dials, and the habits that win points on Friday; every run starts with the data side of this deck.
- ▶ **Week 4:** images in files, read by Dataset classes with transforms that resize, crop and flip; the loader with workers is the difference between a run that finishes and one that does not.
- ▶ **Weeks 5 and 6:** pretrained models and text, where the Dataset returns token ids and the collate function pads them to one length: the same contract, one more argument.

Tensors: **Summary**

- ▶ **Three labels:** shape, dtype, device; every operation needs them to agree, and each disagreement has a message that names the two sides.
- ▶ **Shapes:** batch first; flatten, reshape, unsqueeze relabel, permute moves, stack builds a batch; broadcasting aligns from the right, and [64, 1] against [64] is a silent 64 by 64.
- ▶ **Memory:** elements times bytes; the data is the big thing, so uint8 until the batch, and halve the batch on CUDA out of memory.
- ▶ **Datasets:** two methods, the same shape and dtype every time, check(ds) before the loop; eager when it fits, lazy with workers or a one-off cache when it does not; transforms per sample, statistics from the training split, a seeded split.
- ▶ **Loaders:** sampler, __getitem__, stack, device; shuffle the training loader or the dashboard lies; workers for files, pin_memory for a GPU, the batch size for memory and steps.

Tensors: References & Resources

- [1] PyTorch documentation, `torch.utils.data`: Dataset, DataLoader, samplers, collate, workers.
<https://pytorch.org/docs/stable/data.html>
- [2] PyTorch documentation, Tensor views, broadcasting semantics and `torch.Tensor` dtypes.
<https://pytorch.org/docs/stable/notes/broadcasting.html>
- [3] torchvision documentation, datasets and transforms.
<https://pytorch.org/vision/stable/index.html>
- [4] Paszke, A. et al. (2019). PyTorch: an imperative style, high-performance deep learning library. *NeurIPS* 32.
- [5] LeCun, Y., Cortes, C. and Burges, C. *The MNIST database of handwritten digits*, via OpenML (`mnist_784`); the Day 15 split for every run in this deck.
- [6] NEOAI 2025 (Northern Eurasia Olympiad in AI), task *Cluster Pictures*, Kaggle. The practice task.
- [7] Days 15 and 16 of this course: PyTorch basics; Datasets and DataLoaders; the mini-batch and GPU frames.
- [8] All plots and every number quoted: `slides-builder/scripts/day19_data_figures.py` (loading speed, shuffle, the exact error messages, the memory arithmetic, the practice task's shapes).